

DOCUMENTAÇÃO TÉCNICA · TCC

Pipeline ETL para Análise de Desigualdade Verde Urbana

Universidade Federal Rural do Rio de Janeiro — UFRRJ · Departamento de Computação

AUTOR

Pedro Henrique Rainha de Souza

CURSO

Sistemas de Informação

STATUS

Em desenvolvimento

ÁREA DE ESTUDO

Rio de Janeiro — RJ

DADO PRIMÁRIO

Sentinel-2 SR Harmonizado (2025)

ÍNDICES CALCULADOS

NDVI · MNDWI

1 Visão Geral do Sistema

O sistema implementa um pipeline de Extração, Transformação e Carga (ETL) de imagens de satélite Sentinel-2, processadas na nuvem pelo Google Earth Engine (GEE), com o objetivo de identificar e visualizar a distribuição desigual de vegetação urbana no município do Rio de Janeiro.

O resultado é exibido em um mapa interativo acessível via navegador, com camadas alternáveis de imagem real (RGB), detecção de corpos hídricos (MNDWI) e mapeamento de vegetação urbana (NDVI). As imagens são atualizadas sob demanda por meio de um servidor local em Python (Flask), sem necessidade de recarregar a página.

Escopo atual: A versão documentada contempla o pipeline ETL com GEE, o servidor Flask e a interface Leaflet. Módulos futuros — banco PostGIS, dashboard Streamlit e análise espacial (Gini, Getis-Ord Gi*) — serão documentados em versão subsequente.

2 Arquitetura do Sistema

2.1 Diagrama de Contexto (C4 — Nível 1)

Apresenta os atores externos e sistemas com os quais o pipeline interage, sem detalhar sua estrutura interna.

Sistema principal

Sistema externo

Ator (pessoa)

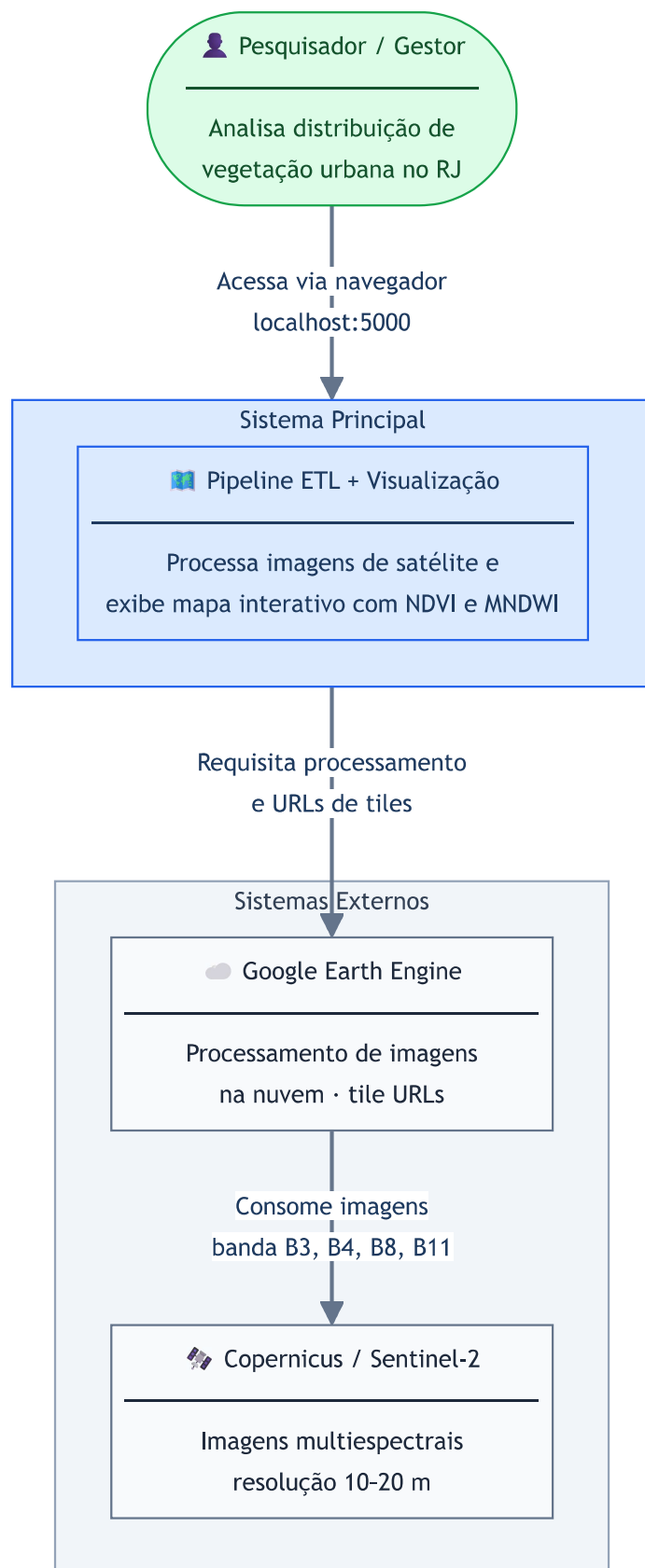


Figura 1 — Diagrama de Contexto C4 (Nível 1): atores e dependências externas do sistema.

2.2 Diagrama de Containers (C4 — Nível 2)

Detalha as unidades de execução independentes do sistema: o servidor Flask (backend), a interface Leaflet (frontend) e o serviço externo do GEE.

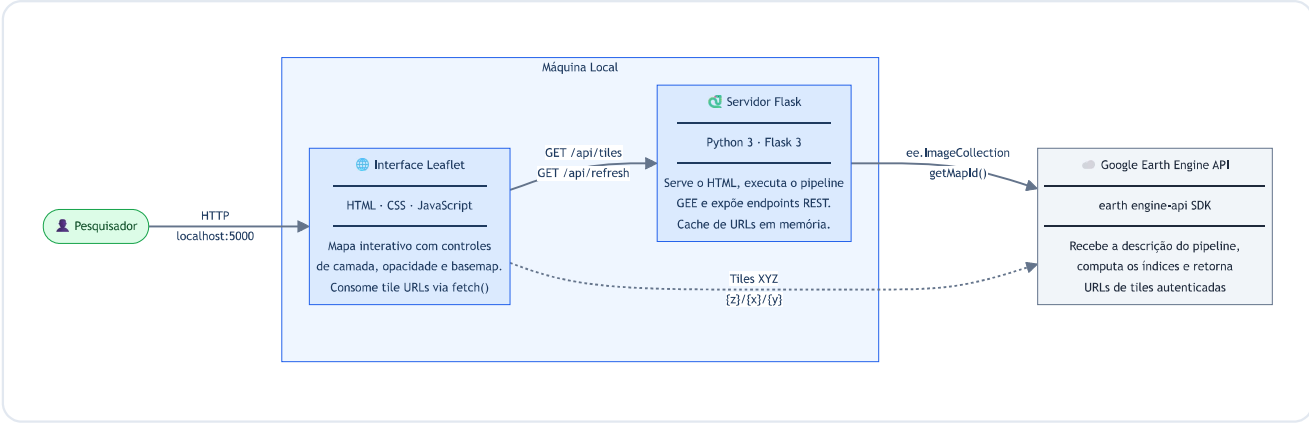


Figura 2 — Diagrama de Containers C4 (Nível 2): componentes de execução e suas comunicações.
Linha sólida = chamada ativa · Linha tracejada = requisição direta de tiles pelo navegador ao GEE.

3 Pipeline ETL

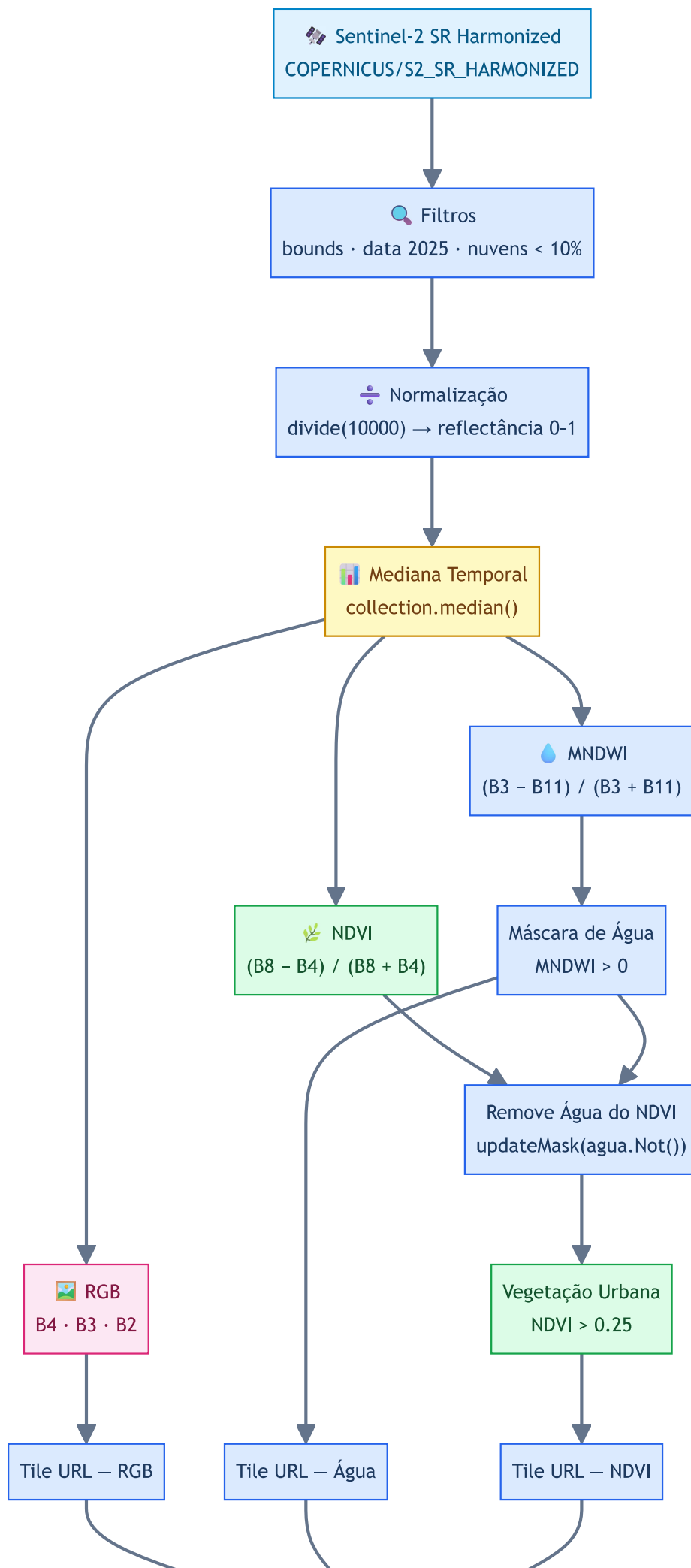
3.1 Descrição das Etapas

Extração: A coleção COPENICUS/S2_SR_HARMONIZED é filtrada por localização (ponto de interesse no RJ), período (2025) e cobertura de nuvens inferior a 10%. Os valores de reflectância são normalizados pela divisão por 10.000.

Transformação: Calcula-se a mediana temporal da coleção para obter uma imagem sem nuvens. Sobre ela são aplicadas as fórmulas de MNDWI (detecção hídrica) e NDVI (vegetação). O NDVI é mascarado nas áreas identificadas como água e filtrado pelo limiar de 0,25 para eliminar ruídos de telhados e solo exposto.

Carga: Cada imagem transformada é publicada como camada de tiles XYZ autenticada via `getMapId()`. As URLs são armazenadas em cache no servidor Flask e entregues ao frontend via API REST.

3.2 Fluxo de Dados



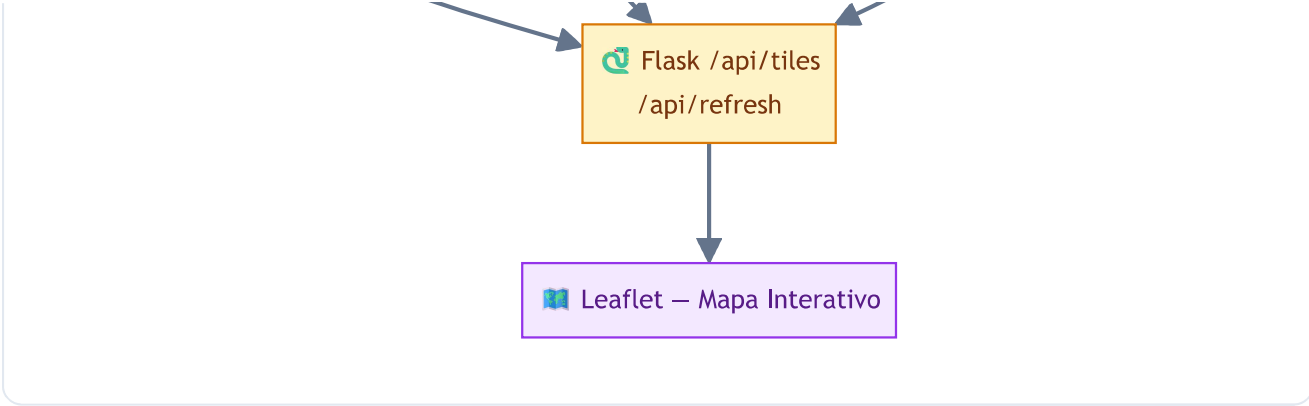


Figura 3 — Fluxo de dados do pipeline ETL: da coleção Sentinel-2 aos tiles renderizados no mapa.

3.3 Fórmulas dos Índices Espectrais

Índice	Fórmula	Bandas Sentinel-2	Finalidade
MNDWI Modified Normalized Difference Water Index	$\frac{(\text{Verde} - \text{SWIR})}{(\text{Verde} + \text{SWIR})}$	B3 (Verde) · B11 (SWIR 1)	Detectar corpos hídricos (rios, canais, lagoas). Valores > 0 indicam água.
NDVI Normalized Difference Vegetation Index	$\frac{(\text{NIR} - \text{Vermelho})}{(\text{NIR} + \text{Vermelho})}$	B8 (NIR) · B4 (Vermelho)	Quantificar vigor da vegetação. Limiar de 0,25 adotado para excluir solo exposto e telhados.

4 Fluxos de Interação

4.1 Carga Inicial do Mapa

Ao acessar localhost:5000, o servidor já está computando as URLs de tiles em uma thread paralela. O frontend realiza polling a cada 1,5 s até as URLs estarem prontas, exibindo um overlay de carregamento no intervalo.

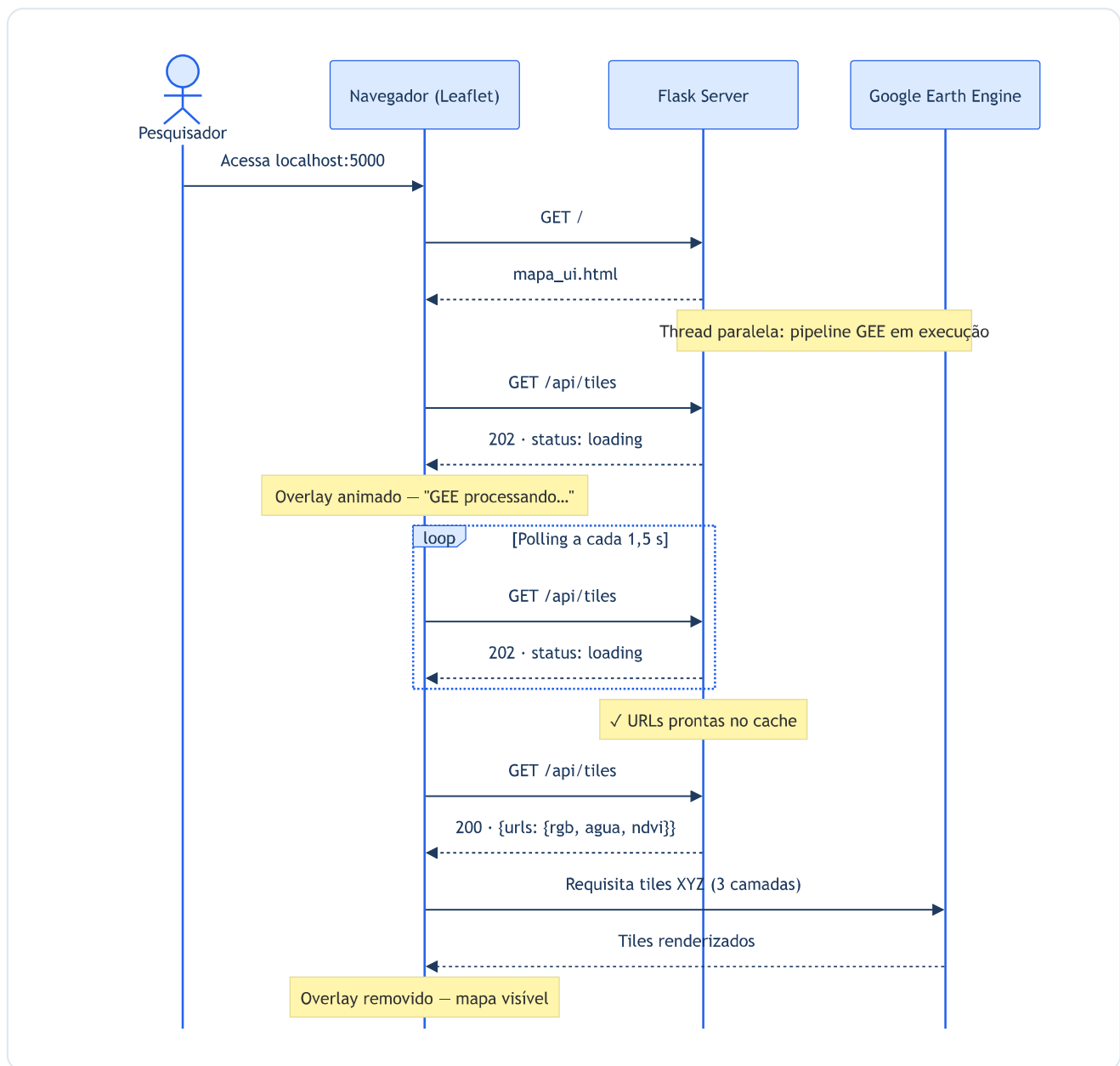


Figura 4 — Sequência de carga inicial: o polling garante que a UI aguarde o pipeline sem bloquear o servidor.

4.2 Atualização de Imagens (Botão "Atualizar")

Ao clicar em "Atualizar Imagens GEE", o frontend chama `/api/refresh`. O servidor reexecuta o pipeline completo e devolve novas URLs. As camadas Leaflet são substituídas em memória, sem recarregar a página.

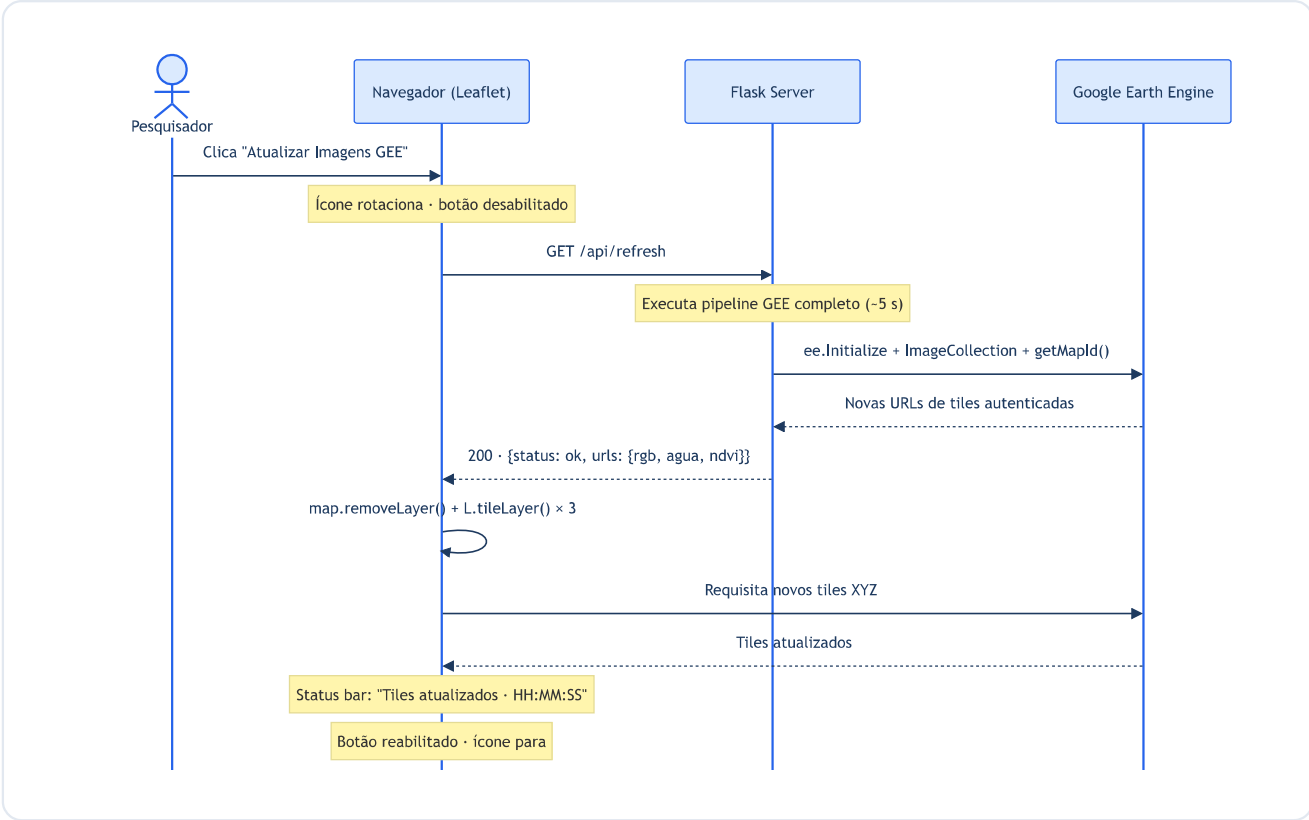


Figura 5 — Sequência de atualização: substituição das camadas Leaflet sem recarregar a página.

5 Stack Tecnológica

Camada	Tecnologia	Versão	Função no sistema
Linguagem	Python	3.14	Orquestração do pipeline ETL e servidor web
Processamento de imagens	Google Earth Engine API	1.7.x	Filtragem, normalização, cálculo de índices e geração de tiles
Servidor web	Flask	3.1	Serve o HTML, expõe endpoints REST, gerencia cache de URLs
Mapa interativo	Leaflet.js	1.9.4	Renderização de tiles XYZ, controles de camada e eventos de mapa
Frontend	HTML5 · CSS3 · JavaScript	—	Interface com sidebar, sliders de opacidade e polling assíncrono (fetch API)
Dado de entrada	Sentinel-2 SR Harmonized	Level-2A	Imagens multiespectrais com correção atmosférica, resolução 10–20 m

6 Endpoints da API REST

Método	Rota	Comportamento	Resposta
GET	/	Serve o arquivo <code>mapa_ui.html</code>	<code>text/html</code>
GET	<code>/api/tiles</code>	Retorna as URLs em cache. Se o pipeline ainda estiver em execução, retorna <code>202 status: loading</code> . O frontend realiza polling até receber <code>200 status: ok</code> .	<code>{"status": "ok", "urls": {"rgb": "...", "agua": "...", "ndvi": "..."}}</code>
GET	<code>/api/refresh</code>	Reexecuta o pipeline ETL completo de forma síncrona e retorna as novas URLs. Atualiza o cache interno. Demora ~5 s (processamento GEE).	<code>{"status": "ok", "urls": {...}}</code> ou <code>{"status": "error", "message": "..."} </code>

Todas as URLs de tiles retornadas pela API são autenticadas e emitidas pelo Google Earth Engine com validade limitada. O frontend as usa diretamente como fonte de tiles XYZ para o Leaflet, sem proxy pelo servidor local.

7 Bibliotecas e Ferramentas

Esta seção detalha cada dependência do sistema: origem, propósito, papel específico no projeto e características relevantes para o contexto acadêmico.

7.1 Python



Python v3.14

Linguagem de programação principal · python.org · Licença PSF (open source)

O QUE É

Linguagem de propósito geral, interpretada e de tipagem dinâmica. Amplamente adotada em ciência de dados, geoprocessamento e automação por sua legibilidade e ecossistema de bibliotecas.

USO NO PROJETO

Orquestra todo o backend: inicializa a conexão com o GEE, executa o pipeline ETL, sobe o servidor web e gerencia o cache de URLs em memória. Utiliza dois módulos da biblioteca padrão sem instalação adicional:

`threading` — permite que o pipeline GEE rode em paralelo sem bloquear o servidor Flask durante a carga inicial.

`webbrowser` — abre o navegador automaticamente ao iniciar o servidor.

CARACTERÍSTICAS RELEVANTES

Interpretado GIL (Global Interpreter Lock) Duck typing PEP 8 Ecossistema pip

7.2 Google Earth Engine API

**earthengine-api** v1.7.xGoogle LLC · developers.google.com/earth-engine · Licença Apache 2.0**O QUE É**

SDK Python para a plataforma Google Earth Engine — um serviço de computação geoespacial em nuvem que disponibiliza petabytes de imagens de satélite e dados geoespaciais públicos, incluindo toda a coleção Copernicus Sentinel-2. O processamento ocorre nos servidores do Google, não localmente.

MODELO DE EXECUÇÃO (LAZY EVALUATION)

As operações Python **não executam imediatamente**. Cada chamada como `ImageCollection.filter()` ou `normalizedDifference()` constrói um grafo de computação (AST) que é enviado ao servidor GEE apenas quando um resultado concreto é solicitado — como em `getMapId()`. Isso permite compor pipelines complexos sem trafegar dados volumosos.

USO NO PROJETO

Extração: `ee.ImageCollection('COPERNICUS/S2_SR_HARMONIZED')` acessa a coleção Sentinel-2 com filtros de localização, data e cobertura de nuvens.

Transformação: `normalizedDifference()` calcula NDVI e MNDWI; `updateMask()` aplica máscaras; `median()` compõe imagem livre de nuvens por estatística temporal.

Carga: `getMapId()` registra a imagem no GEE e retorna uma URL de tiles XYZ autenticada, válida por algumas horas, que o Leaflet consome diretamente.

CARACTERÍSTICAS RELEVANTES

Computação em nuvem Lazy evaluation Grafo computacional Tiles XYZ autenticados Acesso gratuito para pesquisa

7.3 Flask

**Flask** v3.1Pallets Projects · flask.palletsprojects.com · Licença BSD-3-Clause (open source)**O QUE É**

Microframework web para Python. "Micro" significa que o núcleo é mínimo e sem opinião — não impõe ORM, sistema de autenticação ou estrutura de projeto. É extensível via plugins. Amplamente usado para APIs REST, protótipos e aplicações de pequeno a médio porte.

POR QUE FLASK E NÃO FASTAPI OU DJANGO

Flask foi escolhido pela simplicidade: o projeto exige apenas 3 rotas GET sem autenticação de usuário, sem banco de dados e sem schemas complexos. FastAPI adicionaria overhead de tipagem e validação desnecessários; Django seria superdimensionado para o escopo atual.

USO NO PROJETO

GET / — serve o arquivo `mapa_ui.html` via `send_from_directory()`.

GET /api/tiles — retorna o cache de URLs (ou status de carregamento).

GET /api/refresh — reexecuta o pipeline GEE e devolve novas URLs.

A função `jsonify()` do Flask serializa dicionários Python em JSON com os headers HTTP corretos (`Content-Type: application/json`), permitindo que o JavaScript do frontend consuma a resposta com `fetch().json()`.

CARACTERÍSTICAS RELEVANTES

WSGI Roteamento por decorador Servidor de desenvolvimento embutido Extensível via blueprints

7.4 Leaflet.js

**Leaflet.js** v1.9.4Vladimir Agafonkin · leafletjs.com · Licença BSD-2-Clause (open source)

O QUE É

Biblioteca JavaScript de código aberto para mapas interativos no navegador. Com ~42 KB minificados, é a mais popular do gênero. Suporta tiles XYZ, GeoJSON, marcadores, popups, eventos de mapa e é extensível via plugins.

COMO FUNCIONA O SISTEMA DE TILES

O mapa é dividido em uma grade de imagens quadradas de 256×256 px chamadas **tiles**. Cada tile é identificado por três coordenadas {z}/{x}/{y}, onde z é o nível de zoom e x/y são posições na grade. Ao navegar, o Leaflet calcula quais tiles estão visíveis e faz requisições HTTP paralelas para buscá-los. Quando o GEE recebe a requisição de um tile, executa o pipeline apenas para aquela região, retornando uma imagem PNG renderizada.

USO NO PROJETO

`L.map()` — inicializa o mapa no elemento `#map`, centralizado no RJ com zoom 13.

`L.tileLayer(url)` — cria uma camada de tiles a partir de uma URL de template. Usado para o basemap (OSM/Esri/Google) e para as 3 camadas GEE.

`layer.setOpacity()` — controla transparência em tempo real sem recarregar tiles.

`map.removeLayer()` / `addTo(map)` — adiciona e remove camadas dinamicamente.

`layer.bringToBack()` — garante que o basemap fique abaixo das camadas GEE na ordem Z.

CARACTERÍSTICAS RELEVANTES

Tiles XYZ Mobile-friendly Sem dependências externas API de eventos (mousemove, click...) Layer z-ordering

7.5 Dado de Entrada: Sentinel-2 SR Harmonized

**COPERNICUS/S2_SR_HARMONIZED** Level-2AESA / Copernicus Programme · [Acesso público gratuito via GEE](#)

O QUE É

Coleção de imagens do satélite Sentinel-2, desenvolvido pela Agência Espacial Europeia (ESA) no âmbito do programa Copernicus. O produto **SR (Surface Reflectance)** já passou por correção atmosférica — os valores representam a reflectância real da superfície, eliminando interferências da atmosfera. **Harmonized** indica calibração consistente entre as plataformas Sentinel-2A e Sentinel-2B.

ESPECIFICAÇÕES TÉCNICAS

Parâmetro	Valor
Resolução espacial	10 m (B2, B3, B4, B8) · 20 m (B11, B12)
Resolução temporal	Revisita a cada ~5 dias (combinando 2A e 2B)
Bandas totais	13 bandas (VIS, NIR, SWIR, Red Edge)
Escala dos valores brutos	0–10.000 (dividir por 10.000 para reflectância 0–1)
Projeção	UTM / WGS84
Cobertura global	Latitudes entre 56°S e 83°N

BANDAS UTILIZADAS NESTE PROJETO

Banda	Nome	Comprimento de onda	Resolução	Uso
B2	Blue	~490 nm	10 m	Composição RGB
B3	Green	~560 nm	10 m	MNDWI (numerador) · RGB
B4	Red	~665 nm	10 m	NDVI (denominador) · RGB
B8	NIR	~842 nm	10 m	NDVI (numerador)
B11	SWIR 1	~1.610 nm	20 m	MNDWI (denominador)

CARACTERÍSTICAS RELEVANTES

Dados abertos Correção atmosférica L2A Multiespectral Alta revisita temporal Acervo desde 2015

7.6 Fetch API (JavaScript nativo)



Fetch API Web Standard
WHATWG · Nativa nos navegadores modernos · Sem instalação

O QUE É

API nativa do navegador para realizar requisições HTTP de forma assíncrona, substituindo o antigo XMLHttpRequest. Baseada em Promises e compatível com async/await, permite que o JavaScript espere respostas do servidor sem bloquear a interface do usuário.

USO NO PROJETO

Implementa duas interações críticas sem recarregar a página:

Polling inicial: loop de até 30 tentativas chamando `fetch('/api/tiles')` a cada 1,5 s enquanto o GEE processa. Quando o servidor responde com `status: ok`, as URLs são aplicadas ao Leaflet e o overlay de carregamento é removido.

Botão Atualizar: chama `fetch('/api/refresh')` e aguarda a resposta (~10–20 s). Ao receber as novas URLs, substitui as camadas Leaflet em memória — sem nenhum recarregamento de página.

CARACTERÍSTICAS RELEVANTES

Baseada em Promises async/await Sem dependência externa CORS suportado Streaming de respostas

7.7 Mapa de Dependências

Relação entre as bibliotecas e os módulos do sistema que as consomem.

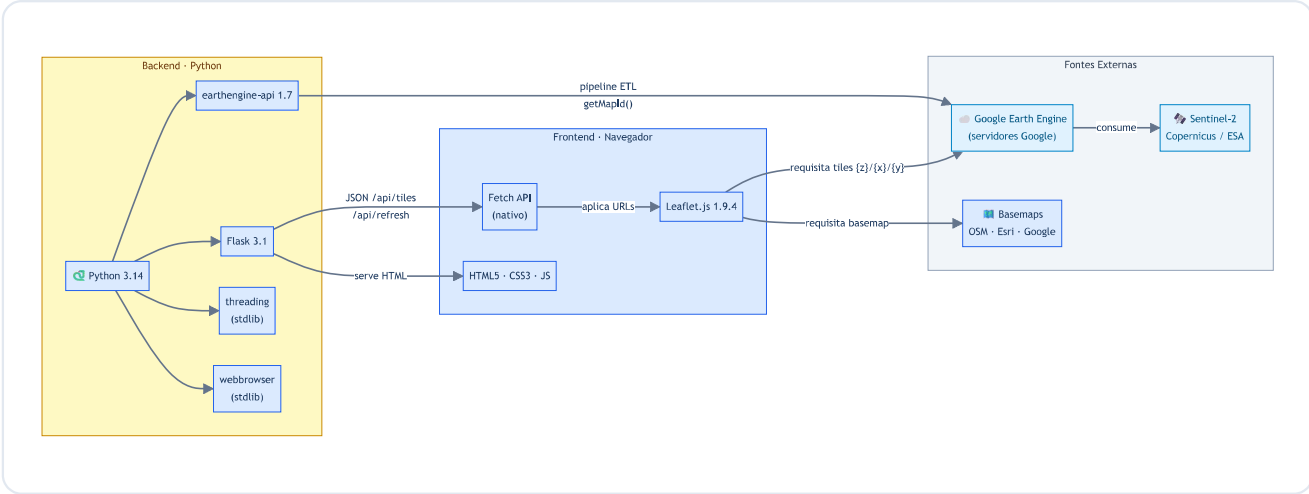


Figura 6 — Mapa de dependências: relação entre bibliotecas, módulos do sistema e serviços externos.